

Абстракция
- аксиома

Енкапсулация

Модулярност

Всичко е обект.

Обектите са отворени.

Класовете са отворени.

Класове

- създават се с ключовата дума `class`
- функциите, дефинирани в тялото на класа, са методи
- променливите, — // — , са клас променливи

`class Vector:`

`def __init__ (self, x, y):`

`self.x = x`

`self.y = y`

`spam = Vector(1.0, 2.0)`

`def length(self):`

`return math.sqrt(self.x**2 + self.y**2)`

`print(spam.length())`

конструктор / инициализатор

инстанцията, върху която се извиква метода

Атрибутите (глен-данните) не се декларират (обектите са отворени)

получаваме новия обект

атрибутите се достъпват само през `self`

извикване метода

```
def _coords(self):
    return (self.x, self.y, self.z)
```

→ protected метод

_име_на_метод - protected методи

- Би трябвало да се използват само от методи на класа и наследяващи го класове

__име_на_метод__ - private методи

- Би трябвало да се ползват само от методи на класа

Unbound methods

```
v1 = Vector(1.0, 2.0, 3.0)
print (Vector.length(v1)) # същото като v1.length
```

Mutable и Immutable

↓
обекти, които променят вътрешното си състояние във времето

↓
обекти, които никога не променят вътрешното си състояние

mutation method - променям стойностите на клен-ранните на моя обект, не връщам нищо

non-mutating method - връщам нов обект

Сравняване на обекти

- проверяваме дали два обекта са равни по стойност с ==
 - проверяваме дали две имена сочат към един и същи обект с is
 - предефиниране метода --eq--, за да сравняваме обекти от даден клас
- ↳ по подразбиране е is

user methods

- предефинират някакъв аспект от поведението на обектите ни

Аритметични оператори

-- add -- (self, other) → self + other
-- sub -- (self, other) → self - other
-- mul -- (self, other) → self * other

...

Преобразуване до стандартни типове

-- float -- (self) → float(обект)

int
complex
bool

Обекти, които могат да бъдат извикани като функции

← имаме инициализатор

def -- call -- (...): дефиниране поведение на обектите си, ако ги извикаме като функции

1) Първо създаваме обект с инициализатора -- init --

2) Използваме -- call --, ако искаме обектът ни да се рърни като функция

getattr / setattr

v1 = Vector(x, y, z)

getattr(v1, 'y') → взима елемент-данната

#2

setattr(v1, 'z', 5) → променя стойността на елемент-данната

getattr(v1, 'z')

#5

Статични методи
@staticmethod
....

Класови методи
@classmethod
...

Деструктор

def --del--(self):
→ del method (не е точно destructor)
• затваря именовани връзки или файлове
• изпраща съобщения до някъде и други...

Property method

@property → декоратор
- може да се използва, ако искаме друг метод да се преструва на property
↓ метода ни
да се държи като атрибут; не ^{го}викам с ()
Ако искаме да променим стойността на метод, който се преструва на атрибут → използваме setter-ите.

@property
def func(self):

...
@func.setter
def func(self, ...):
...
като подадем обект.func() викаме това

- Функциите са първокласни обекти
- Методите са атрибути на класа
- Класовете са динамични
- self е явен

